

2η Προγραμματιστική Εργασία-1η Φάση

Τεχνητή Νοημοσύνη

Student: Ιωάννης Μπουρίτης, Εμμανουήλ Θωμάς Χατζάκης
Student ID: 202130173,2021030061

Περιεχόμενα

1	Εισαγωγή	2
2	Περιγραφή Περιβαλλόντων	2
2.1	CartPole-v1	2
2.2	MountainCar-v0	2
3	Wrappers	2
3.1	Τροποποίηση ανταμοιβής σε one-step lookahead	2
3.2	Μη-στασιμότητα περιβάλλοντος	3
4	Αλγόριθμοι	3
4.1	Κοινό πλαίσιο τοπικής αναζήτησης	3
4.2	Random Agent	3
4.3	Hill Climbing Agent	3
4.4	Randomized Hill Climbing Agent	4
4.5	Simulated Annealing Agent	4
4.5.1	Μηχανισμός Λήψης Απόφασης και Θερμοκρασία	4
4.5.2	Cooling Schedule & Συγχρονισμός Χρόνου	4
5	Αποτελέσματα	5
5.1	Μέση αθροιστική ανταμοιβή	5
5.2	Ανταμοιβή ανά επεισόδιο	7
6	Συμπεράσματα	8
7	Πηγές	8

1 Εισαγωγή

Η παρούσα εργασία εξετάζει την απόδοση αλγορίθμων τοπικής αναζήτησης σε δύο κλασικά περιβάλλοντα ενισχυτικής μάθησης: `CartPole-v1` και `MountainCar-v0`. Σε αντίθεση με αναζήτηση που αξιοποιεί γνωστές δυναμικές εξισώσεις, εδώ ο πράκτορας δεν έχει πρόσβαση στο μοντέλο μετάβασης του περιβάλλοντος. Αντ' αυτού, αξιολογεί τις υποψήφιες ενέργειες μέσω προσομοίωσης ενός βήματος (*one-step lookahead*), χρησιμοποιώντας `deep copy` του περιβάλλοντος.

Επιπλέον, τα περιβάλλοντα παρουσιάζουν μη-στασιμότητα: οι φυσικές παράμετροί τους μεταβάλλονται κατά τη διάρκεια κάθε επεισοδίου μέσω του `NSClassicControlWrapper`. Ο συνδυασμός περιορισμένου ορίζοντα αναζήτησης, θορυβώδους σήματος ανταμοιβής και μη-στατικής δυναμικής δημιουργεί ένα ιδιαίτερα δύσκολο setup, στο οποίο αξιολογούμε τους αλγορίθμους `Hill Climbing`, `Randomized Hill Climbing` και `Simulated Annealing`, συγκρίνοντάς τους με έναν τυχαίο πράκτορα ως γραμμή βάσης.

2 Περιγραφή Περιβαλλόντων

2.1 `CartPole-v1`

Το `CartPole-v1` είναι ένα κλασικό πρόβλημα ελέγχου από το `OpenAI Gymnasium`. Ένας πόλος είναι συνδεδεμένος μέσω άρθρωσης σε ένα καρότσι που κινείται σε ευθεία χωρίς τριβές. Ο πράκτορας πρέπει να εφαρμόζει διακριτή δύναμη προς τα αριστερά ή προς τα δεξιά, έτσι ώστε ο πόλος να παραμένει σε όρθια θέση για όσο το δυνατόν περισσότερα χρονικά βήματα.

Ο χώρος καταστάσεων είναι συνεχής και τετραδιάστατος (θέση καροτσιού, ταχύτητα, γωνία πόλου, γωνιακή ταχύτητα). Ο χώρος ενεργειών αποτελείται από δύο διακριτές επιλογές: ώθηση αριστερά και ώθηση δεξιά. Η ανταμοιβή είναι $+1$ για κάθε βήμα κατά το οποίο ο πόλος παραμένει ισορροπημένος. Το επεισόδιο τερματίζεται όταν ο πόλος αποκλίνει υπερβολικά ή το καρότσι εξέλθει εκτός ορίων, ή όταν συμπληρωθεί το μέγιστο πλήθος βημάτων (truncation).

2.2 `MountainCar-v0`

Το `MountainCar-v0` αφορά ένα αυτοκίνητο με ανεπαρκή ισχύ, το οποίο πρέπει να συσσωρεύσει ορμή μέσω επαναλαμβανόμενης κίνησης μπρος-πίσω, ώστε να υπερβεί έναν λόφο. Η βασική δυσκολία είναι ότι η άμεση κίνηση προς τον στόχο δεν αρκεί.

Ο χώρος καταστάσεων είναι διδιάστατος (θέση, ταχύτητα). Ο χώρος ενεργειών περιλαμβάνει τρεις επιλογές: ώθηση αριστερά, καμία ενέργεια και ώθηση δεξιά. Η ανταμοιβή είναι -1 για κάθε βήμα, και το επεισόδιο θεωρείται επιτυχές όταν η θέση φτάσει τουλάχιστον 0.5 με μη αρνητική ταχύτητα.

3 Wrappers

3.1 Τροποποίηση ανταμοιβής σε *one-step lookahead*

Οι wrappers εφαρμόζονται αποκλειστικά στο `deep-copied planning environment` κατά τη φάση της προσομοίωσης και όχι στο πραγματικό περιβάλλον. Σκοπός τους είναι να τροποποιούν το σήμα ανταμοιβής που παρατηρεί ο πράκτορας κατά το *one-step lookahead*.

Συγκεκριμένα, σε κάθε βήμα της προσομοίωσης προστίθεται ένας τυχαίος όρος στο εύρος $[-0.5, 0.5]$ στην ανταμοιβή που επιστρέφει το βασικό περιβάλλον. Αυτό εισάγει μικρή διαφοροποίηση στις τιμές που παρατηρούν οι αλγόριθμοι, χωρίς να δημιουργεί ουσιαστική πληροφορία για το ποια ενέργεια είναι πραγματικά καλύτερη.

Στο CartPole-v1, η βασική ανταμοιβή είναι πάντα +1 για μη τερματικά βήματα, οπότε με την προσθήκη θορύβου το σήμα κινείται απλώς στο $[0.5, 1.5]$. Αντίστοιχα, στο MountainCar-v0 η βασική ανταμοιβή είναι σταθερά -1, και ο θόρυβος δεν αρκεί για να αποκαλύψει ποια ενέργεια συμβάλλει στη μακροπρόθεσμη επίτευξη του στόχου.

Επιπλέον, οι wrappers ορίζουν μία συνάρτηση `reward()`, η οποία υπολογίζει μια πιο «κατευθυντική» ανταμοιβή με βάση τη φυσική κατάσταση του συστήματος. Ωστόσο, η συνάρτηση αυτή δεν χρησιμοποιείται από τους υλοποιημένους πράκτορες, οι οποίοι αξιοποιούν μόνο τη μέθοδο `step()`.

3.2 Μη-στασιμότητα περιβάλλοντος

Στο πειραματικό setup εφαρμόζεται επίσης ο `NSClassicControlWrapper`, ο οποίος καθιστά τα περιβάλλοντα μη-στάσιμα. Στο CartPole, η μάζα του πόλου αυξάνεται σε κάθε βήμα μέσω `ContinuousScheduler`, ενώ η βαρύτητα μεταβάλλεται περιοδικά (ανά 3 βήματα) με τυχαία διακύμανση (`PeriodicScheduler + RandomWalk`). Στο MountainCar, τόσο η βαρύτητα όσο και η δύναμη του κινητήρα αυξάνονται συνεχώς.

Αυτή η μη-στασιμότητα σημαίνει ότι η δυναμική του περιβάλλοντος αλλάζει εντός του ίδιου επεισοδίου. Ένας πράκτορας που βασίζεται σε σταθερούς κανόνες one-step αξιολόγησης δεν μπορεί να προσαρμοστεί επαρκώς στις μεταβολές αυτές.

4 Αλγόριθμοι

4.1 Κοινό πλαίσιο τοπικής αναζήτησης

Όλοι οι αλγόριθμοι κληρονομούν από την κλάση `LocalSearchAgent`, η οποία παρέχει το κοινό πλαίσιο εκτέλεσης. Σε κάθε απόφαση γίνεται deep copy του planning environment και προσομοίωση ενός βήματος για κάθε υποψήφια ενέργεια. Για το CartPole, η αξιολόγηση επιβραβεύει μεγαλύτερο t (επιβίωση), ενώ για το MountainCar τιμωρεί μεγάλο t (αργή επίτευξη στόχου). Ωστόσο, ο χρονικός όρος είναι ίδιος για όλες τις ενέργειες σε δεδομένο βήμα, οπότε δεν προσφέρει διαφοροποίηση μεταξύ των επιλογών.

Η μέθοδος `find_best_route()` διαχειρίζεται τον κύκλο εκτέλεσης: σε κάθε βήμα δημιουργεί deep copy του περιβάλλοντος για το planning, συγχρονίζει τον χρόνο (`sim_env.t = env.t`) και τροφοδοτεί την επιλεγμένη ενέργεια στο πραγματικό περιβάλλον.

4.2 Random Agent

Ο τυχαίος πράκτορας (`TestingAgent`) επιλέγει κάθε ενέργεια ομοιόμορφα στην τύχη, χωρίς κανενός είδους lookahead ή αξιολόγηση. Αποτελεί τη γραμμή βάσης σύγκρισης απέναντι στην οποία αξιολογούνται οι υπόλοιποι αλγόριθμοι.

4.3 Hill Climbing Agent

Ο Hill Climbing Agent εξετάζει όλες τις διαθέσιμες ενέργειες μέσω one-step lookahead και επιλέγει ντετερμινιστικά εκείνη με τη μέγιστη τιμή αξιολόγησης. Ενέργειες που οδηγούν σε τερματική κατάσταση αποκλείονται εξ ορισμού. Αν όλες οι επιλογές οδηγούν σε τερματισμό, ο πράκτορας επιστρέφει -1, οπότε ο μηχανισμός εκτέλεσης καταλήγει σε τυχαία επιλογή.

Πρόκειται για έναν αυστηρά greedy αλγόριθμο χωρίς εξερεύνηση. Επειδή όμως οι μη τερματικές ενέργειες δεν διαφέρουν ουσιαστικά ως προς τη βασική ανταμοιβή, η επιλογή της «καλύτερης» ενέργειας καθορίζεται πρακτικά από τον τυχαίο θόρυβο του wrapper.

4.4 Randomized Hill Climbing Agent

Ο Randomized Hill Climbing Agent λειτουργεί παρόμοια με τον κλασικό Hill Climbing, αλλά διατηρεί ένα κατώφλι βελτίωσης μέσω της τρέχουσας τιμής `current_value`. Από τις διαθέσιμες ενέργειες, ο αλγόριθμος συγκρατεί μόνο εκείνες που εμφανίζουν τιμή υψηλότερη από την `current_value` και επιλέγει τυχαία ανάμεσά τους. Αν δεν υπάρχει καμία βελτιωτική ενέργεια, επιστρέφει -1 και ο μηχανισμός εκτέλεσης επιλέγει τυχαία.

Η βασική διαφορά από τον HC είναι η εισαγωγή στοχαστικότητας μεταξύ βελτιωτικών γειτόνων. Παρ' όλα αυτά, το πρόβλημα της έλλειψης ουσιαστικού σήματος παραμένει, καθώς οι «βελτιώσεις» καθορίζονται κυρίως από θόρυβο.

4.5 Simulated Annealing Agent

Ο Simulated Annealing Agent χρησιμοποιεί πιθανότητα αποδοχής χειρότερων λύσεων που εξαρτάται από τη θερμοκρασία, επιτρέποντας έτσι την αποφυγή τοπικών βελτίσεων. Οι παράμετροι που επιλέχθηκαν είναι `initial_temp = 10.0`, `cooling_rate = 0.99` και `min_temp = 1e-3`.

Με γεωμετρική ψύξη, η θερμοκρασία μειώνεται αργά και φτάνει στο κατώφλι της πέριπου μετά από 916 βήματα. Στην πράξη, για μικρά επεισόδια ο αλγόριθμος παραμένει σχεδόν πλήρως εξερευνητικός, ενώ για μεγαλύτερα επεισόδια καταλήγει να λειτουργεί πιο ντετερμινιστικά.

4.5.1 Μηχανισμός Λήψης Απόφασης και Θερμοκρασία

Ο αλγόριθμος αξιολογεί όλες τις διαθέσιμες ενέργειες μέσω one-step lookahead. Παράλληλα, διατηρεί την τρέχουσα τιμή V_{current} , η οποία αντιστοιχεί στην αξιολόγηση της κατάστασης στο προηγούμενο χρονικό βήμα (αποθηκευμένη στο `self.current_value`). Σε κάθε απόφαση επιλέγεται τυχαία μία ενέργεια ως γειτονική κατάσταση, με τιμή V_{next} . Η διαφορά ορίζεται ως:

$$\Delta V = V_{\text{next}} - V_{\text{current}}$$

Αν $\Delta V \geq 0$, η ενέργεια γίνεται αποδεκτή αυτόματα (βελτίωση). Αν $\Delta V < 0$, η χειρότερη επιλογή δεν απορρίπτεται απόλυτα, αλλά μπορεί να γίνει αποδεκτή με πιθανότητα που ακολουθεί την κατανομή Boltzmann:

$$P(\text{accept}) = e^{\frac{\Delta V}{T}} \quad (1)$$

όπου T είναι η τρέχουσα θερμοκρασία του αλγορίθμου. Σημειώνεται ότι η σύγκριση γίνεται με την τιμή της τρέχουσας θέσης (προηγούμενο βήμα), και όχι με τη βέλτιστη διαθέσιμη ενέργεια --- αυτό αποτελεί το κλασικό κριτήριο αποδοχής της προσομοιωμένης ανόπτησης.

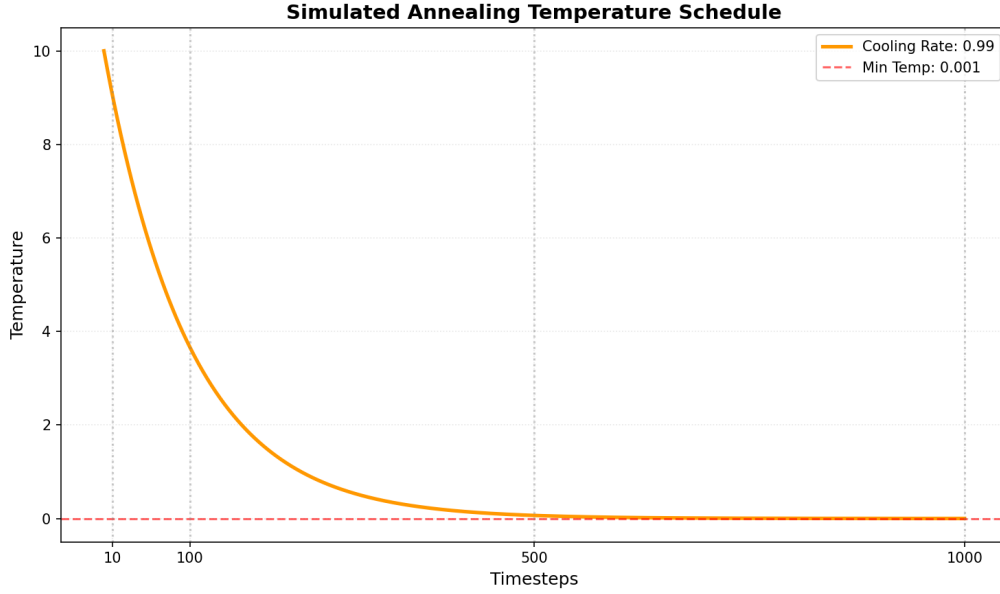
4.5.2 Cooling Schedule & Συγχρονισμός Χρόνου

Η θερμοκρασία μειώνεται εκθετικά σε κάθε βήμα:

$$T_{\text{new}} = \max(T_{\text{old}} \cdot \alpha, T_{\text{min}})$$

με $\alpha = 0.99$ και $T_{\text{min}} = 10^{-3}$. Η εκθετική ψύξη καθορίζει τη μετάβαση από εξερεύνηση σε εκμετάλλευση, όπως φαίνεται στο Σχήμα 1.

Επιπλέον, εισήχθη μηχανισμός συγχρονισμού χρόνου κατά τη διάρκεια του planning. Η τρέχουσα χρονική στιγμή μεταφέρεται στο προσομοιωμένο περιβάλλον με την εντολή `sim_env.t = getattr(env, 't', 0)`. Χωρίς αυτόν τον συγχρονισμό, η `calculate_value`



Σχήμα 1: Εκθετική μείωση της θερμοκρασίας ($T_0 = 10.0$, $\alpha = 0.99$, $T_{\min} = 10^{-3}$) συναρτήσει των χρονικών βημάτων κατά την προσομοιωμένη ανόπτηση.

δεν μπορούσε να υπολογίσει σωστά τον χρονικό όρο επιβίωσης στο CartPole, στερώντας από τον πράκτορα το κίνητρο για μακροχρόνια διατήρηση ισορροπίας.

Στην επιλογή υπερπαραμέτρων παρατηρήθηκε ότι υπερβολικά υψηλές αρχικές τιμές θερμοκρασίας οδηγούσαν σε συμπεριφορά τυχαίου περιπάτου, με αποτέλεσμα πρόωρο τερματισμό. Η τελική επιλογή $T_0 = 10$ εξισορροπεί εξερεύνηση και εκμετάλλευση για τα εύρη επεισοδίων που εξετάστηκαν.

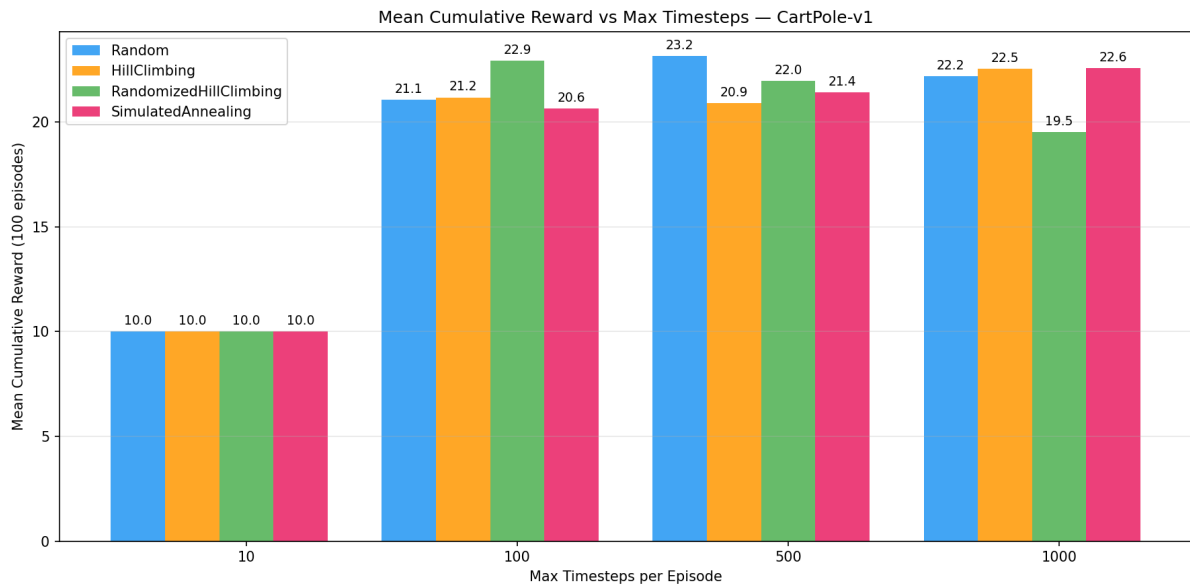
5 Αποτελέσματα

5.1 Μέση αθροιστική ανταμοιβή

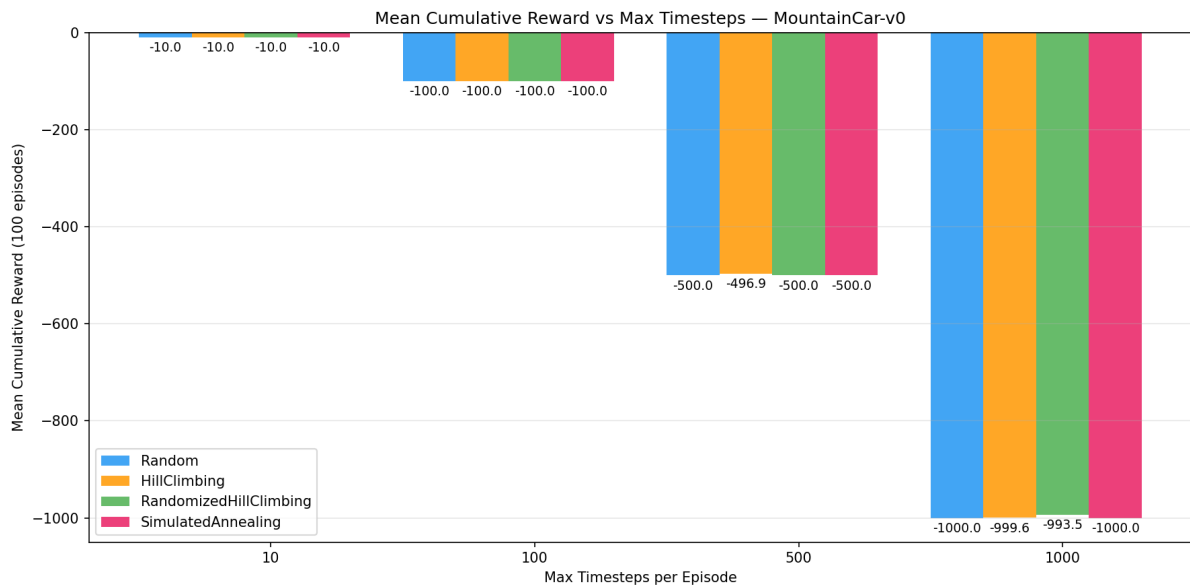
Για κάθε συνδυασμό αλγορίθμου και περιβάλλοντος, τρέξαμε 100 επεισόδια για $\text{max_timesteps} \in \{10, 100, 500, 1000\}$ και υπολογίσαμε τη μέση αθροιστική ανταμοιβή. Τα αποτελέσματα φαίνονται στα Σχήματα 2 και 3.

CartPole-v1. Για μικρό αριθμό βημάτων ($\text{max_timesteps} = 10$), όλοι οι αλγόριθμοι αποδίδουν ταυτόσημα (≈ 10), καθώς εξαντλούν το επεισόδιο. Για $\text{max_timesteps} \in \{100, 500\}$, παρατηρούνται μικρές αποκλίσεις της τάξης των 1--2 μονάδων, εντός του εύρους τυχαίας διακύμανσης. Αξιοσημείωτη εξαίρεση αποτελεί το $\text{max_timesteps} = 1000$, όπου ο Randomized Hill Climbing εμφανίζει χαμηλότερη μέση ανταμοιβή (19.5) σε σχέση με τους υπόλοιπους (≈ 22 --22.6). Αυτό οφείλεται στο ότι ο αλγόριθμος επιστρέφει -1 (τυχαία επιλογή) όταν καμία ενέργεια δεν ξεπερνά το current_value --- σε μεγάλα επεισόδια, η θύρα αυτή ανοίγει πιο συχνά και η τυχαία επιλογή αυξάνει τον κίνδυνο τερματισμού.

MountainCar-v0. Τα αποτελέσματα είναι πιο ξεκάθαρα: κανένας αλγόριθμος δεν επιτυγχάνει τον στόχο, και όλοι εξαντλούν το μέγιστο πλήθος βημάτων, λαμβάνοντας ουσιαστικά τη μέγιστη δυνατή ποινή ($-\text{max_timesteps}$). Παρατηρούνται δύο μικρές αποκλίσεις: ο Hill Climbing στα 500 βήματα (-496.9 αντί -500) και ο Randomized Hill Climbing στα 1000 βήματα (-993.5 αντί -1000). Και οι δύο αποδίδονται στον τυχαίο θόρυβο και όχι σε πραγματική βελτίωση στρατηγικής.



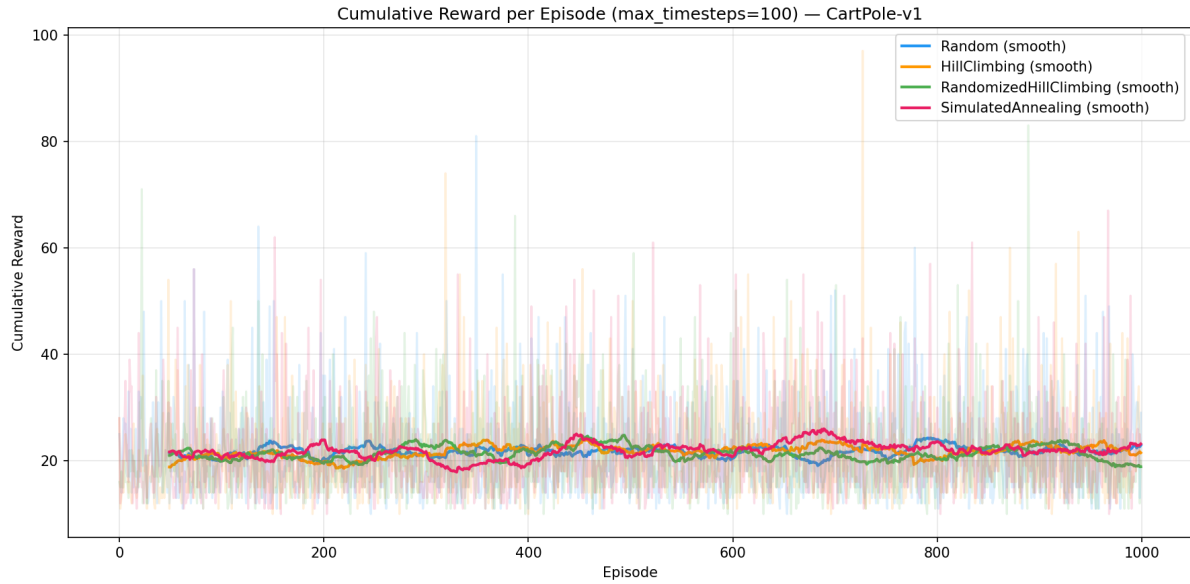
Σχήμα 2: Μέση αθροιστική ανταμοιβή (100 επεισόδια) ανά αλγόριθμο και max_timesteps --- CartPole-v1.



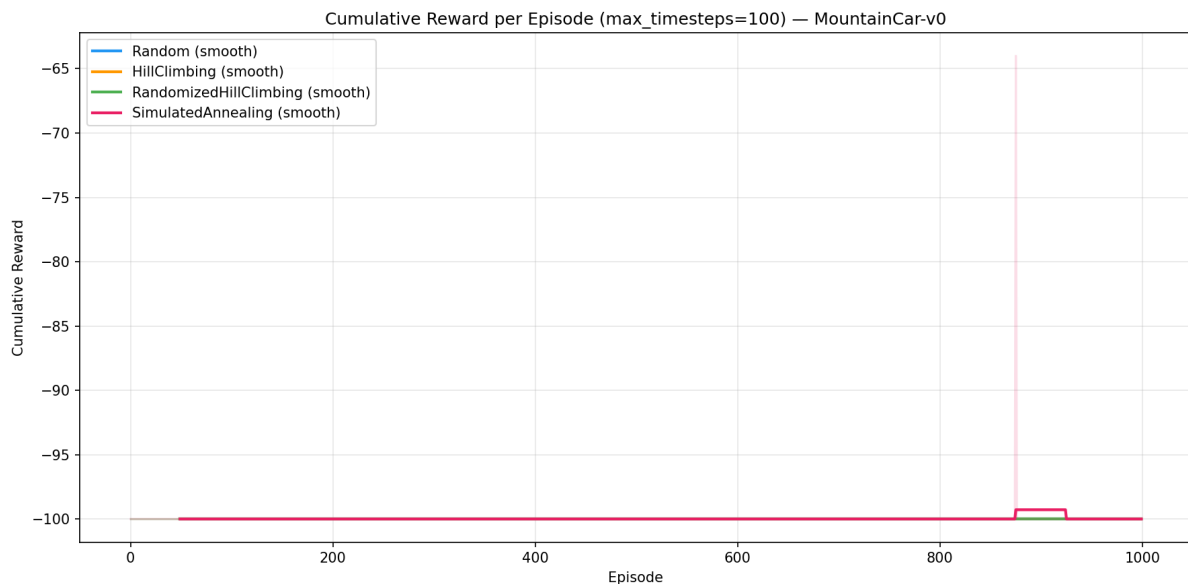
Σχήμα 3: Μέση αθροιστική ανταμοιβή (100 επεισόδια) ανά αλγόριθμο και max_timesteps --- MountainCar-v0.

5.2 Ανταμοιβή ανά επεισόδιο

Για $\text{max_timesteps} = 100$ και 1000 επεισόδια, παρακολουθήσαμε την αθροιστική ανταμοιβή ανά επεισόδιο προκειμένου να εξετάσουμε τη συμπεριφορά των πρακτόρων καθώς το περιβάλλον μεταβάλλεται (Σχήματα 4 και 5).



Σχήμα 4: Αθροιστική ανταμοιβή ανά επεισόδιο ($\text{max_timesteps} = 100$, 1000 επεισόδια) --- CartPole-v1. Οι έντονες γραμμές αντιστοιχούν σε gliding average παραθύρου 50 επεισοδίων.



Σχήμα 5: Αθροιστική ανταμοιβή ανά επεισόδιο ($\text{max_timesteps} = 100$, 1000 επεισόδια) --- MountainCar-v0.

CartPole-v1. Οι καμπύλες όλων των αλγορίθμων παραμένουν σταθερές και αλληλοεπικαλυπτόμενες, χωρίς καμία ένδειξη βελτίωσης ή χειροτέρευσης με την πάροδο των επεισοδίων. Αυτό είναι αναμενόμενο: οι αλγόριθμοι δεν έχουν μνήμη ούτε μηχανισμό μάθησης, οπότε κάθε επεισόδιο αντιμετωπίζεται ανεξάρτητα.

MountainCar-v0. Όλες οι καμπύλες συμπίπτουν σε σταθερή τιμή -100 , που αντιστοιχεί στο μέγιστο δυνατό penalty για 100 βήματα. Κανένας πράκτορας δεν καταφέρνει να φτάσει στον στόχο. Αξιοσημείωτη εξαίρεση αποτελεί μια απότομη μεταβολή που εμφανίζεται για τον Simulated Annealing γύρω στο επεισόδιο 880, κατά την οποία η ανταμοιβή αποκλίνει προσωρινά από -100 . Η ανωμαλία αυτή αποδίδεται στη στοχαστική φύση του αλγορίθμου: υψηλή θερμοκρασία σε συνδυασμό με ευνοϊκή τυχαία αρχικοποίηση του επεισοδίου μπορεί να οδηγήσει σε ασυνήθη σειρά αποδεκτών κινήσεων. Δεν πρόκειται για μάθηση, αλλά για τυχαία διακύμανση.

6 Συμπεράσματα

Τα πειράματα δείχνουν ξεκάθαρα ότι, στο συγκεκριμένο setup, κανένας από τους αλγορίθμους τοπικής αναζήτησης δεν καταφέρνει να υπερβεί ουσιαστικά την τυχαία επιλογή. Ο περιορισμός σε one-step lookahead, σε συνδυασμό με σταθερές βασικές ανταμοιβές και θόρυβο στο planning environment, αφαιρεί κάθε αξιόπιστο σήμα που θα επέτρεπε τη διάκριση μεταξύ καλών και κακών ενεργειών.

Ιδιαίτερα ενδιαφέρον είναι το γεγονός ότι ο Randomized Hill Climbing εμφανίζει χειρότερη απόδοση από τον τυχαίο πράκτορα στο CartPole για μεγάλα επεισόδια ($\text{max_timesteps} = 1000$). Αυτό συμβαίνει επειδή η ύπαρξη κατωφλίου βελτίωσης (current_value) σε συνδυασμό με θορυβώδες σήμα αυξάνει τη συχνότητα τυχαίων επιλογών, οι οποίες με τη σειρά τους αυξάνουν τον κίνδυνο τερματισμού.

7 Πηγές

Hill climbing algorithm
Hill Climbing in Artificial Intelligence

Gymnasium (Gymnasium.farama.org)
Simulated Annealing
Simulated Annealing (GeeksforGeeks)